

Task #1: Greedy Algorithms: Dijkstra's Algorithm

Assume below is result of Dijkstra

Pre = [-, A, F, F, B, A]

Dist = [0, 2, 4, 5, 5, 1]

Shortest path for each and distance for each path

A to B:

A to E:

A to C:

A to F:

A to D:

Task #2: Coin-collecting Robot Problem

1) Input:

C[] =		1	2	3	4	5	6	7
1	0	0	0	1	0	0	0	0
2	0	0	0	0	1	0	0	0
3	0	1	1	0	0	0	0	1
4	1	0	0	0	1	0	0	0
5	0	0	0	1	0	0	0	0
6	1	0	0	0	0	0	0	0
7	0	1	0	0	0	0	1	0

Your work out:

F[] =		1	2	3	4	5	6	7
1								
2								
3								
4								
5								
6								
7								

Max for Cell (2,3) = _____

Max for Cell (4,7) = _____

Max for Cell (7,4) = _____

Max for Cell (7,7) = _____

2) Input:

C[] =		1	2	3	4	5	6	7
1	0	1	0	0	0	0	0	0
2	0	0	1	0	0	0	1	0
3	1	0	0	1	0	0	0	1
4	0	0	0	1	1	0	0	0
5	0	1	0	1	0	0	0	0
6	0	0	0	0	0	0	0	0
7	0	0	1	1	0	0	0	0

Your work out:

F[] =		1	2	3	4	5	6	7
1								
2								
3								
4								
5								
6								
7								

Max for Cell (3,4) = _____

Max for Cell (4,3) = _____

Max for Cell (5,7) = _____

Max for Cell (7,5) = _____

3) Input:

C[] =		1	2	3	4	5	6	7
1	0	2	0	0	3	0	0	0
2	0	0	1	0	0	0	1	0
3	2	0	0	0	3	0	0	3
4	0	0	1	3	1	0	0	0
5	0	2	0	3	0	0	0	0
6	1	0	0	0	0	0	0	0
7	0	0	3	0	0	2	0	0

Your work out:

F[] =		1	2	3	4	5	6	7
1								
2								
3								
4								
5								
6								
7								

Max for Cell (2,4) = _____

Max for Cell (4,3) = _____

Max for Cell (7,5) = _____

Max for Cell (7,7) = _____

Task #3: Fibonacci Numbers

Write simple Java code for dynamic programming solution for Fibonacci	Explain why your solution is DP
--	--

Task #4: Java code for finding the length of the longest strictly increasing subsequence from a given array

<pre>public class LongestIncreasingSubsequence { public static int lengthOfLIS(int[] nums) { if (nums.length == 0) return 0; int[] dp = new int[nums.length]; for (int i = 0; i < dp.length; i++) { dp[i] = 1; // Each element is an increasing subsequence of at least length 1. } for (int i = 1; i < nums.length; i++) { for (int j = 0; j < i; j++) { if (nums[i] > nums[j]) { dp[i] = Math.max(dp[i], dp[j] + 1); } } } // Find the maximum value in dp array int maxLength = 0; for (int length : dp) { maxLength = Math.max(maxLength, length); } return maxLength; // The length of the longest increasing subsequence. } public static void main(String[] args) { int[] nums = {10, 9, 2, 5, 3, 7, 101, 18}; System.out.println(lengthOfLIS(nums)); // Output: 4 } }</pre>	Why this solution is DP solution?
---	--